

Práctico AYED1 - Sala 2 - 04-11-2021

Práctico 7 - Ejercicio 2

Calcular la suma de un arreglo.

Especificación:

```
Const N: Int, A: array[0,N) of Int;  
Var suma: Int;  
{ P:  $N \geq 0$  }  
S  
{ Q: suma =  $\langle \sum i : 0 \leq i < N : A.i \rangle$  }
```

Solución intuitiva:

```
Const N: Int, A: array[0,N) of Int;  
Var suma, i: Int;  
{ P:  $N \geq 0$  }  
  suma, i := 0, 0;  
  do  $i \neq N \rightarrow$   
    suma, i := suma + A.i, i + 1  
  od  
{ Q: suma =  $\langle \sum i : 0 \leq i < N : A.i \rangle$  }
```

Derivación:

Voy a necesitar un ciclo para calcular la sumatoria.

Para encontrar el invariante vamos a usar la técnica de reemplazo de constante por variable.

Introducimos una nueva variable $k : \text{Int}$ y proponemos el siguiente invariante:

$$\text{INV} \equiv \text{suma} = \langle \sum i : 0 \leq i < k : A.i \rangle \wedge 0 \leq k \leq N$$

- esta parte es la que resulta de hacer el cambio de constante por variable en la postcondición.

Observación: acá podría haber reemplazado el 0 en lugar del N.

- esta parte es un fortalecimiento que hacemos.

La parte $k \leq N$ me sirve para asegurarme de que la cuantificación está bien definida (o sea, no se pasa del arreglo), y la parte $k \geq 0$ ya veremos pero nos va a servir (y además vale).

Además, proponemos la siguiente guarda:

$$B \equiv k \neq N$$

De esta manera, queda garantizado que $\text{INV} \wedge \neg B \Rightarrow Q$.

Demostración: Supongamos INV y $\neg B$ y veamos Q:

$$\text{suma} = \langle \sum i : 0 \leq i < N : A.i \rangle$$

$$\equiv \{ \text{hip. } \neg B, \text{ o sea } k = N \}$$

$$\text{suma} = \langle \sum i : 0 \leq i < k : A.i \rangle$$

$$\equiv \{ \text{hip. INV} \}$$

True

Veamos cómo es el programa hasta el momento:

```
Const N: Int, A: array[0,N) of Int;
Var suma, k: Int;
{ P:  $N \geq 0$  }
  S0;
{ INV }      // de esta manera, el requisito de que la pre del do implique al invariante se convierte en  $INV \Rightarrow INV$ 
do  $k \neq N \rightarrow$ 
  {  $INV \wedge B$  }
  S1
  { INV }      // esta terna se desprende del requisito de que el cuerpo del ciclo preserve el invariante
od
{ Q:  $\text{suma} = \langle \sum i : 0 \leq i < N : A.i \rangle$  }
```

Sólo nos falta derivar S0 (inicialización) y S1 (cuerpo del ciclo). Para ambos, tenemos sus respectivas pre y postcondiciones.

Inicialización: Repasemos la terna que hay que derivar:

```
{ P:  $N \geq 0$  }
  S0
{ INV:  $\text{suma} = \langle \sum i : 0 \leq i < k : A.i \rangle \wedge 0 \leq k \leq N$  }
```

Como siempre, quiero aplicar la ley del mínimo esfuerzo.

Lo primero sería ver si $P \Rightarrow INV$, en cuyo caso S0 sería **skip**.

Pero está claro que no vale $P \Rightarrow INV$, porque P no dice nada de k ni de suma.

La siguiente sentencia a probar es la asignación. Podemos probar S0 como una asignación de la forma:

$\text{suma}, k := E, F$

Ahora, nuestras incógnitas son las expresiones E y F. Debemos elegir las tales que valga:

$$P \Rightarrow wp.(suma, k := E, F).INV$$

Además, E y F deben ser expresiones válidas de nuestro lenguaje de programación (por ej no se puede cuantificaciones).

Entonces, supongamos que vale P ($N \geq 0$) y veamos la wp:

$$\begin{aligned} & wp.(suma, k := E, F).INV \\ \equiv & \{ \text{def. wp} \} \\ & E = \langle \sum i : 0 \leq i < F : A.i \rangle \wedge 0 \leq F \leq N \\ \equiv & \{ \text{elegimos } \underline{F = 0} \} \\ & E = \langle \sum i : 0 \leq i < 0 : A.i \rangle \wedge \underline{0 \leq 0 \leq N} \\ \equiv & \{ \text{hip.} \} \\ & E = \langle \sum i : 0 \leq i < 0 : A.i \rangle \\ \equiv & \{ \text{rango vacío} \} \\ & E = 0 \\ \equiv & \{ \text{elijo } \underline{E = 0} \} \\ & \text{True} \end{aligned}$$

Observación: muchas veces elegiremos inicializar tratando de forzar rangos vacíos o rangos unitarios que me permitan deshacerme de las expresiones cuantificadas.

Cuerpo del ciclo: Repasemos la terna que hay que derivar:

$$\begin{aligned} & \{ INV \wedge B: suma = \langle \sum i : 0 \leq i < k : A.i \rangle \wedge 0 \leq k \leq N \wedge k \neq N \} \\ & S1 \\ & \{ INV: suma = \langle \sum i : 0 \leq i < k : A.i \rangle \wedge 0 \leq k \leq N \} \end{aligned}$$

Ley del mínimo esfuerzo:

- ¿Puedo elegir que S1 sea **skip**? ¡No! Si fuera **skip**, el ciclo nunca terminaría
Si bien vale que " $INV \wedge B \Rightarrow INV$ ", también S1 debe cumplir la siguiente terna para la cota:

$$\{ INV \wedge B \wedge t = T \} S1 \{ t < T \}$$

(la cota decrece). **skip** no cumple esto.

Entonces siempre al derivar el cuerpo del ciclo tengo que tener presente también la cota, y que ésta decrezca.

En este caso, la función de cota es

$$t = N - k$$

Con esta cota, podemos ver que vale el requisito:

- $INV \Rightarrow t \geq 0$ (Ejercicio: demostrar, es muy facil ya que el invariante dice $k \leq N$)

Si S1 no es skip, que sea una asignación.

Además nos vamos a adelantar un poco más y vamos a plantear que a k le asignamos k+1.

S1 va a ser de la forma:

$$\text{suma}, k := E, k + 1$$

Debo encontrar E tal que vale:

$$INV \wedge B \Rightarrow wp.(\text{suma}, k := E, k + 1).INV$$

Supongamos $INV \wedge B$ y veamos la wp:

$$wp.(\text{suma}, k := E, k + 1).INV$$

$$\equiv \{ \text{def. wp} \}$$

$$E = \langle \sum i : 0 \leq i < k + 1 : A.i \rangle \wedge \underline{0 \leq k + 1 \leq N}$$

$$\equiv \{ 0 \leq k + 1 \text{ por hip } 0 \leq k . k + 1 \leq N \text{ por hip } k \leq N \text{ y } k \neq N. \}$$

$$E = \langle \sum i : 0 \leq i < k + 1 : A.i \rangle$$

$$\equiv \{ \text{lógica} \}$$

$$E = \langle \sum i : 0 \leq i < k \vee i = k : A.i \rangle$$

$$\equiv \{ \text{partición de rango + rango unitario} \}$$

$$E = \langle \sum i : 0 \leq i < k : A.i \rangle + A.k$$

$$\equiv \{ \text{hip} \}$$

$E = \text{suma} + A.k$
 $\equiv \{ \text{elijo } \underline{E = \text{suma} + A.k} \}$
 True

¡Listo! El programa final queda:

```

Const N: Int, A: array[0,N) of Int;
Var suma, k: Int;
{ P:  $N \geq 0$  }
  suma, k := 0, 0 ;
  do  $k \neq N \rightarrow$ 
    suma, k := suma + A.k, k + 1
  od
{ Q: suma =  $\langle \sum i : 0 \leq i < N : A.i \rangle$  }
  
```

Pregunta: ¿Qué hubiera pasado si hacía el siguiente reemplazo de constante por variable?

$INV \equiv \text{suma} = \langle \sum i : k \leq i < N : A.i \rangle \wedge 0 \leq k \leq N$

(reemplazo el 0 en lugar de N)

- Para que valga $INV \wedge \neg B \Rightarrow Q$, la guarda tendría que ser: $B \equiv k \neq 0$.
- La inicialización sería: $\text{suma}, k := 0, N$.
- El cuerpo del ciclo sería: $\text{suma}, k := \text{suma} + A.(k-1), k := k - 1$.
- **Ejercicio: chequear que la derivación da esto!**

Práctico 7 - Ejercicio 1a

Dado $N \geq 0$, calcular el menor entero x tal que $x^3 + x \geq N$.

Especificación:

Const $N : \text{Int}$;

Var $x : \text{Int}$;

{ $P: N \geq 0$ }

S

{ $Q: x = \langle \text{Min } i : i^3 + i \geq N : i \rangle$ }

Ayuda: En lugar de usar los cuantificadores de mínimo y máximo, escribir la postcondición como una conjunción.

Lo pensamos un poco:

" x es el menor entero tal que $x^3 + x \geq N$." es lo mismo que decir:

" x cumple $x^3 + x \geq N$ " y además

"todos los que son mas chicos que x no lo cumplen"

Usando esto, la postcondición sería:

{ $Q: x^3 + x \geq N \wedge \langle \forall i : 0 \leq i < x : i^3 + i < N \rangle$ }

Derivación: Necesitamos un ciclo. En este caso nuestra postcondición es una conjunción. Vamos a probar con la técnica de tomar términos de una conjunción.

Está técnica me dice que una parte de la conjunción será la negación de la guarda (por ende debe una expresión válida en mi lenguaje de programación), y la otra parte será el invariante.

En nuestro caso,

$$B \equiv x^3 + x < N$$

$$INV \equiv \langle \forall i : 0 \leq i < x : i^3 + i < N \rangle$$

De esta manera, queda garantizado que $INV \wedge \neg B \Rightarrow Q$ (de hecho, son equivalentes).

El programa tendrá la siguiente forma:

Const N : Int ;

Var x ; Int ;

{ P: $N \geq 0$ }

S0 ;

{ INV }

do $x^3 + x < N \rightarrow$ // esta guarda ya me regala una posible cota: $N - (x^3 + x)$. (también anda $N - x$)

{ $INV \wedge B$ }

S1

{ INV }

od

{ Q: $x^3 + x \geq N \wedge \langle \forall i : 0 \leq i < x : i^3 + i < N \rangle$ }

Falta derivar inicialización y cuerpo del ciclo (S0 y S1).

Inicialización: Probamos con $x := E$. Debe valer $P \Rightarrow wp.(x := E).INV$.

Supongamos P y veamos la wp:

$wp.(x := E).INV$.

$\equiv \{ \text{wp de } := \}$
 $\langle \forall i : 0 \leq i < E : i^3 + i < N \rangle$
 $\equiv \{ \text{elijo } \mathbf{E = 0} \text{ forzando rango vacío} \}$
 True

Cuerpo: Probamos con $x := x + 1$. (teniendo la cota en mente también.)

Debe valer $\text{INV} \wedge B \Rightarrow \text{wp.}(x := x + 1).\text{INV}$. Supongamos $\text{INV} \wedge B$ y veamos la wp:

$\text{wp.}(x := x + 1).\text{INV}$
 $\equiv \{ \text{wp de } := \}$
 $\langle \forall i : 0 \leq i < x + 1 : i^3 + i < N \rangle$
 $\equiv \{ \text{lógica} \}$
 $\langle \forall i : 0 \leq i < x \vee i = x : i^3 + i < N \rangle$
 $\equiv \{ \text{partición y rango unitario} \}$
 $\langle \forall i : 0 \leq i < x : i^3 + i < N \rangle \wedge x^3 + x < N$
 $\equiv \{ \text{hip. INV} \}$
 $x^3 + x < N$
 $\equiv \{ \text{hip. B} \}$
 True

¡Listo! El programa final queda:

```

Const N : Int ;
Var x ; Int ;
{ P: N ≥ 0 }
x := 0 ;
do x^3 + x < N →
  x := x + 1

```

od

$\{ Q: x^3 + x \geq N \wedge \langle \forall i: 0 \leq i < x: i^3 + i < N \rangle \}$